

TheLastKeep最后的要塞-架构设计

项目主要技术栈

本项目是 Qt Widgets + C++面向对象的2D塔防游戏。

Qt Widgets 就是Qt用来做桌面窗口界面的组件库。

整体架构

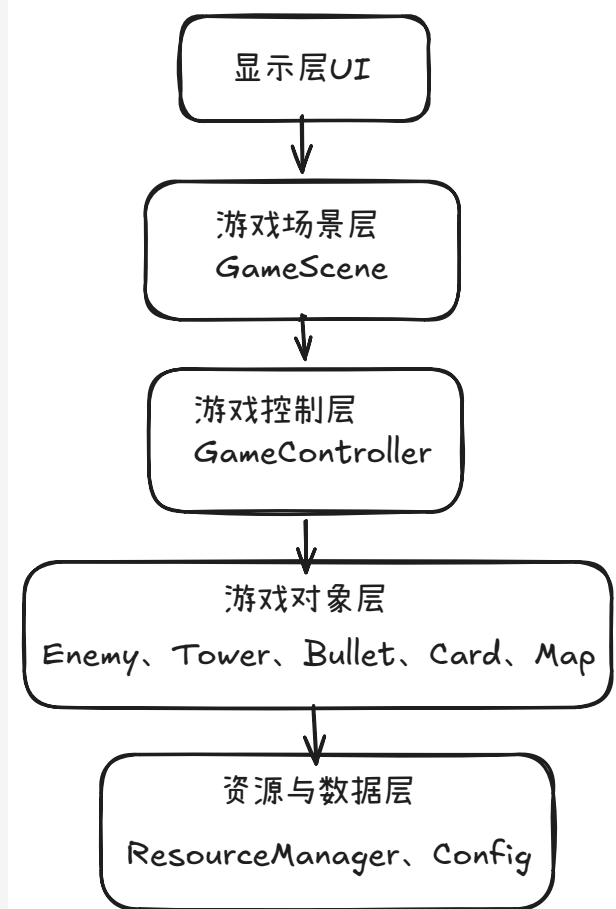
游戏整体上分为5层设计，最底层是游戏中各类资源的images和音乐，以及游戏数据。

游戏对象层，主要是游戏核心类，关卡中的敌人、Tower、Bullet、Card、Map、Buff等等。

游戏控制层，主要是游戏核心逻辑。

游戏场景层，是游戏场景的设计和实现，主要是Qt

最顶层是显示层UI。



项目结构

```
TheLastKeep/  
├─ CMakeLists.txt  
├─ main.cpp  
├─ resources.qrc  
├─ assets/  
│   └─ images/
```

```
| | | | map/
| | | | enemies/
| | | | towers/
| | | | bullets/
| | | | cards/
| | | |   ui/
| | | |   sounds/
|
| | | | ui/
| | | | | MainWindow.h
| | | | | MainWindow.cpp
| | | | | StartMenu.h
| | | | | StartMenu.cpp
| | | | | GameOverPanel.h
| | | | | GameOverPanel.cpp
|
| | | | core/
| | | | | GameScene.h
| | | | | GameScene.cpp
| | | | | GameController.h
| | | | | GameController.cpp
| | | | | GameState.h
| | | | | GameConfig.h
|
| | | | map/
| | | | | GameMap.h
| | | | | GameMap.cpp
| | | | | Tile.h
| | | | | Tile.cpp
| | | | | WayPoint.h
| | | | | WayPoint.cpp
|
| | | | entity/
| | | | | Enemy.h
| | | | | Enemy.cpp
| | | | | Tower.h
| | | | | Tower.cpp
| | | | | Bullet.h
| | | | | Bullet.cpp
| | | | | Castle.h
| | | | | Castle.cpp
|
| | | | card/
| | | | | Card.h
| | | | | Card.cpp
| | | | | CardManager.h
```

```
|   └─ CardManager.cpp
|
└─ manager/
    ├── WaveManager.h
    ├── WaveManager.cpp
    ├── ResourceManager.h
    ├── ResourceManager.cpp
    ├── CollisionManager.h
    └─ CollisionManager.cpp
```

这是VERSION 0.1的结构设计，后期有可能会做一些修改。

核心类

- MainWindow
- GameScene
- GameController
- GameMap
- Enemy
- Tower
- Bullet
- WaveManager
- CardManager

1. MainWindow: 主窗口

负责:

```
开始游戏
进入游戏场景
暂停游戏
重新开始
退出游戏
```

2. GameScene: 游戏画布

继承:

```
class GameScene : public QGraphicsScene
```

负责显示和事件：

- 显示地图
- 显示敌人
- 显示塔
- 显示子弹
- 处理鼠标点击建塔
- 处理卡牌选择
- 每帧刷新游戏对象

核心函数可以设计成：

```
class GameScene : public QGraphicsScene
{
    Q_OBJECT

public:
    explicit GameScene(QObject *parent = nullptr);

    void startGame();
    void pauseGame();
    void resumeGame();
    void resetGame();

protected:
    void mousePressEvent(QGraphicsSceneMouseEvent *event) override;

private slots:
    void updateGame();

private:
    QTimer *m_gameTimer;
    GameController *m_controller;
};
```

3. GameController：游戏规则控制器

这个类非常重要，主要负责规则。

金币多少
城堡生命值多少
当前第几波
是否胜利
是否失败
是否进入卡牌选择阶段
建塔是否允许
扣钱
加钱
怪物死亡奖励

可以这样设计：

```
class GameController
{
public:
    GameController();

    void start();
    void update();

    bool canBuildTower(int cost) const;
    void spendGold(int amount);
    void addGold(int amount);

    void damageCastle(int damage);
    bool isGameOver() const;
    bool isVictory() const;

    int gold() const;
    int castleHp() const;
    int score() const;
    int currentWave() const;

private:
    int m_gold;
    int m_castleHp;
    int m_score;
    int m_currentWave;
    bool m_gameOver;
    bool m_victory;
};
```

这样后面 UI 要显示金币、血量、波次，都问 `GameController` 要。

4. GameMap：地图和路线

塔防游戏必须有地图。

可以先设计成网格地图，比如：

地图宽 12 格
地图高 8 格
每格 64×64 像素

每个格子有类型：

```
enum class TileType
{
    Road,          // 怪物道路
    Buildable,     // 可以建塔
    Blocked,       // 不可放置
    Castle         // 城堡
};
```

地图类：

```
class GameMap
{
public:
    GameMap();

    TileType tileAt(int row, int col) const;
    bool canBuildAt(int row, int col) const;

    QPointF tileToScenePos(int row, int col) const;
    QPoint scenePosToTile(QPointF pos) const;

    QVector<QPointF> enemyPath() const;

private:
    QVector<QVector<TileType>> m_tiles;
```

```
    QVector<QPointF> m_path;  
};
```

敌人就沿着 `m_path` 走。

5. WayPoint: 路径点

敌人路线可以用一串点表示。

比如：

```
m_path = {  
    QPointF(0, 320),  
    QPointF(160, 320),  
    QPointF(160, 128),  
    QPointF(480, 128),  
    QPointF(480, 512),  
    QPointF(768, 512)  
};
```

敌人每次向下一个点移动。

```
class Enemy : public QGraphicsPixmapItem  
{  
public:  
    Enemy(const QVector<QPointF> &path);  
  
    void updateMove();  
  
    bool reachedCastle() const;  
    void takeDamage(int damage);  
    bool isDead() const;  
  
private:  
    QVector<QPointF> m_path;  
    int m_currentTargetIndex;  
  
    int m_hp;  
    int m_speed;  
};
```

游戏对象设计

1. Enemy 敌人

属性：

血量
速度
奖励金币
当前路径点
是否死亡
是否到达城堡

函数：

```
void updateMove();  
void takeDamage(int damage);  
bool isDead() const;  
bool reachedCastle() const;
```

可以先做一种敌人：

普通哥布林
HP = 100
Speed = 2
Reward = 10

后面再扩展：

重甲兵：血厚慢
狼骑兵：血少快
巫师：抗魔
Boss：高血量

2. Tower 防御塔

属性：

攻击范围
攻击力
攻击间隔
攻击冷却
建造价格
目标敌人

函数：

```
void updateAttack(QList<Enemy*> enemies);  
Enemy* findTarget(QList<Enemy*> enemies);  
Bullet* createBullet(Enemy* target);
```

第一版先做一种塔：

弓箭塔
伤害 25
范围 160
攻击间隔 800ms
价格 50

后面再扩展：

法师塔：高伤害慢速
炮塔：范围伤害
冰霜塔：减速
圣光塔：对亡灵增伤

3. Bullet 子弹

属性：

速度
伤害
目标敌人
是否命中

函数：

```
void updateMove();  
bool hitTarget() const;
```

第一版做“追踪子弹”，最简单：

每一帧朝目标敌人移动
距离足够近就判定命中

这样不用处理复杂弹道。

卡牌系统设计

设计思路：

每过一波，暂停游戏
随机出现三张卡牌
玩家选择一张
卡牌效果永久生效或立即生效
继续下一波

卡牌类型：

```
enum class CardType  
{  
    TowerDamageUp,  
    TowerRangeUp,  
    TowerAttackSpeedUp,  
    GoldBonus,  
    CastleHeal,  
    EnemySlow,  
    BulletSpeedUp  
};
```

Card 类：

```

class Card
{
public:
    Card(CardType type, QString name, QString description);

    void apply(GameController *controller);

    QString name() const;
    QString description() const;
    CardType type() const;

private:
    CardType m_type;
    QString m_name;
    QString m_description;
};

```

CardManager:

```

class CardManager
{
public:
    QList<Card> generateThreeCards();
    void applyCard(const Card &card, GameController *controller);

private:
    QList<Card> m_cardPool;
};

```

游戏主循环

使用 `QTimer`。

```

m_gameTimer = new QTimer(this);
connect(m_gameTimer, &QTimer::timeout, this, &GameScene::updateGame);
m_gameTimer->start(16);

```

每一帧做这些事：

1. 更新敌人移动
2. 更新塔攻击
3. 更新子弹移动
4. 检测敌人是否死亡
5. 检测敌人是否到达城堡
6. 检测当前波次是否结束
7. 如果波次结束，进入卡牌选择
8. 刷新金币、血量、波次 UI
9. 判断胜利或失败

伪代码：

```
void GameScene::updateGame()
{
    if (m_controller->isPaused()) {
        return;
    }

    m_waveManager->update();

    for (Enemy *enemy : m_enemies) {
        enemy->updateMove();
    }

    for (Tower *tower : m_towers) {
        Bullet *bullet = tower->updateAttack(m_enemies);
        if (bullet) {
            addBullet(bullet);
        }
    }

    for (Bullet *bullet : m_bullets) {
        bullet->updateMove();
    }

    checkEnemyState();
    checkBulletState();
    checkWaveState();
    updateUI();
}
```

波次系统

WaveManager 负责刷怪。

```
class WaveManager
{
public:
    WaveManager();

    void startWave(int waveIndex);
    void update();

    bool isWaveFinished() const;
    bool hasMoreWaves() const;

private:
    int m_currentWave;
    int m_spawnedCount;
    int m_totalEnemyCount;
    int m_spawnInterval;
    int m_spawnCooldown;
};
```

第一版可以这样：

第 1 波：5 个普通敌人
第 2 波：8 个普通敌人
第 3 波：10 个普通敌人 + 2 个重甲敌人
第 4 波：15 个敌人
第 5 波：Boss

数据流设计

一次完整游戏流程：

```
玩家点击开始游戏
↓
MainWindow 创建 GameScene
↓
GameScene 初始化地图、控制器、波次管理器
↓
WaveManager 开始刷怪
↓
```

Enemy 沿路径移动
↓
Tower 自动寻找 Enemy
↓
Tower 生成 Bullet
↓
Bullet 命中 Enemy
↓
Enemy 扣血死亡
↓
GameController 增加金币和分数
↓
一波结束
↓
CardManager 生成三张卡
↓
玩家选择强化
↓
下一波继续
↓
城堡血量为 0: 失败
全部波次结束: 胜利

分工

陈思睿: 核心战斗逻辑

负责:

Enemy
Tower
Bullet
Collision / hit detection
伤害、攻击、死亡逻辑

对应文件:

entity/Enemy.*
entity/Tower.*

```
entity/Bullet.*
manager/CollisionManager.*
```

刘嘉航：地图与波次系统

负责：

```
GameMap
Tile
WayPoint
WaveManager
敌人路线
刷怪节奏
```

对应文件：

```
map/GameMap.*
map/Tile.*
map/WayPoint.*
manager/WaveManager.*
```

鱼浩琳：界面与卡牌系统

负责：

```
MainWindow
GameScene
UI 显示
Card
CardManager
游戏开始/暂停/结束
```

对应文件：

```
ui/MainWindow.*
core/GameScene.*
card/Card.*
card/CardManager.*
```

不过 `GameScene` 会和所有模块交互，建议组长或者代码整合能力强的人负责。

加入特色

第一阶段跑通后，再加：

1. 波次系统
2. 每波结束三选一卡牌
3. 多种塔
4. 多种敌人
5. 开始菜单
6. 胜利/失败界面
7. 图片资源
8. 背景音乐和音效

第三阶段做展示优化

最后再做：

1. 中世纪风格 UI
2. 城堡、骑士、哥布林、弓箭塔素材
3. 卡牌插画
4. 暂停菜单
5. 说明界面
6. 代码注释
7. 报告截图
8. 打包 exe

重要基础参数

窗口大小: **1024 × 768**
地图格子: **64 × 64**
地图尺寸: **16 列 × 10 行**
初始金币: **150**
城堡血量: **10**
弓箭塔价格: **50**
弓箭塔伤害: **25**
弓箭塔范围: **160**
弓箭塔攻击间隔: **800ms**
普通敌人血量: **100**
普通敌人速度: **2**
普通敌人奖励: **10 金币**
每帧刷新: **16ms**

目前计划的开发顺序

1. 新建 Qt CMake 项目
2. 写 **GameScene**, 显示黑色背景
3. 写 **GameMap**, 画出道路和可建造区域
4. 写 **Enemy**, 让敌人沿路径移动
5. 写 **Tower**, 点击格子生成塔
6. 写 **Bullet**, 塔自动攻击敌人
7. 写 **GameController**, 加入金币和血量
8. 写 **WaveManager**, 定时刷怪
9. 写 **CardManager**, 每波结束三选一
10. 替换图片和美化 UI

最关键的是第 3 到第 6 步。